National Tsing Hua University
Department of Electrical Engineering

Undergraduate Project Report

Incremental Placement Optimization Beyond Detailed Placement: Simultaneous Gate Sizing, Buffering, and Cell Relocation

Advisor	Prof. Wai-Kei Mak
Members	Ting-Yao Huang & Cheng-Jui Hsu
Research period	17 March 2025 to 17 September 2025 (6 months)

Abstract

Commercial placement-optimization flows usually apply gate sizing, buffer insertion and cell relocation as separate, sequential stages. Because the three are tightly coupled, optimizing them one at a time narrows the search space and tends to stop at a local optimum.

This project, developed for Problem C of the 2025 CAD Contest ("Incremental Placement Optimization Beyond Detailed Placement"), presents an integrated framework that applies buffer insertion, gate sizing, cell relocation and wirelength optimization in a closed loop. The loop keeps iterating while the contest score S improves. Every iteration produces a replayable ECO changelist and a legal DEF, as the contest requires.

With equal weights ($\alpha = \beta = \gamma = 1$) on the six public benchmarks, the flow reduces total negative slack (TNS) by 91.8%–100% and half-perimeter wirelength (HPWL) by 7.9%–75.2%. Total power falls by 80.0%–91.3% on three designs but rises on the other three, which exposes the timing–power trade-off of aggressive buffering. The work received an Honorable Mention in the domestic round of the CAD Contest.

Contents

1 Introduction **1**

2 Problem Formulation **1**

2.1 Allowed transformations 1

2.2 Inputs, outputs and legality 2

2.3 Scoring 2

3 Proposed Method **3**

3.1 Overall flow 3

3.2 Strategy selection 3

3.3 Buffer insertion 3

3.4 Legalization and changelist generation 4

3.5 Gate sizing 5

3.6 Cell relocation 5

3.7 Wirelength optimization 5

4 Experimental Results **5**

5 Conclusion **7**

6 Plan Management and Teamwork **7**

6.1 Plan management 7

6.2 Teamwork 7

1 Introduction

Incremental placement optimization refines power, performance and area (PPA) after an initial placement, and it is a key stage of modern physical design. Conventional flows are iterative. They apply gate sizing, buffering and cell relocation one transformation at a time, and run static timing analysis (STA) after each one to confirm the gain.

This paradigm is slow and suboptimal. Each new transformation must work around every earlier one, so a fix on one critical path can degrade another path or break placement legality. A more global, coordinated way of improving PPA is needed.

The 2025 CAD Contest Problem C, set by NVIDIA [7], targets exactly this gap. Contestants receive a legal seed placement and may relocate cells, resize gates and insert buffers. The output must stay legal and is scored on timing, power, wirelength, cell displacement and runtime together.

This report makes three contributions:

1. A closed-loop flow that applies buffer insertion, gate sizing, cell relocation and wirelength optimization in sequence, and repeats the loop while the contest score S improves.
2. A row-occupancy legalizer that places every newly inserted buffer on a legal site near its driver, so each changelist command can be replayed on the original DEF.
3. A weight-aware strategy selector that chooses how aggressively to optimize timing from the testcase weights (α, β, γ) and the seed's TNS.

Section 2 defines the problem and the scoring. Section 3 describes the method. Section 4 reports results on the six public benchmarks, and Section 5 concludes. Section 6 covers project management.

2 Problem Formulation

2.1 Allowed transformations

The contest allows three transformations. They interact: for example, a newly inserted buffer occupies placement sites and so shrinks the room left for relocating other cells.

Cell relocation. Moving connected cells closer shortens the wire between them, which lowers its resistance R and capacitance C. That cuts the Elmore wire delay (about $0.69\,RC$) and the switching power $\alpha C V_{DD}^2 f$. Faster transitions also shorten short-circuit time. In a dense design, though, pulling one pair together pushes other pairs apart, so the moves must be balanced. Placement is NP-hard even in simplified forms [1].

Gate sizing. Each cell can be swapped for a library variant with the same function but a different drive strength or threshold-voltage flavour. Upsizing a driver lowers its output resistance and speeds up the stage it drives. It also raises the input capacitance that the upstream stage sees, which slows that stage, and it increases leakage and switching power. Discrete gate sizing under timing constraints is NP-hard [3].

Buffer insertion. A buffer, or a pair of inverters, splits a long or high-fanout net into shorter segments. This shields a critical sink from a heavy load and sharpens slow transitions, at the cost of extra area, leakage and the buffer's own switching power. Inserting buffers optimally at minimum cost is NP-hard [4].

2.2 Inputs, outputs and legality

- **Inputs.** A gate-level Verilog netlist, the seed placement as DEF and as Bookshelf files (`.nodes`, `.nets`, `.pl`, `.scl`), and ASAP7 library files (`.lib` timing and power models, `.lef` geometry).
- **Outputs.** An updated DEF, plus an ECO changelist of `size_cell` and `insert_buffer` commands in PrimeTime syntax. The evaluator replays the changelist on the original DEF and checks that the result matches the submitted DEF; invalid commands are skipped.
- **Legality.** Every cell must sit on a valid site with no overlap. IO ports and fixed macros must not move, and the netlist must stay functionally equivalent and consistent with the library.

2.3 Scoring

OpenROAD [5] measures TNS, total power and wirelength. Each is normalized against the seed, defined as the average over all teams' legal solutions:

$$TNS_{norm} = \frac{TNS_{sol} - TNS_{seed}}{|TNS_{seed}|}, \tag{1}$$

$$Power_{norm} = \frac{Power_{sol} - Power_{seed}}{Power_{seed}}, \tag{2}$$

$$WL_{norm} = \frac{WL_{sol} - WL_{seed}}{WL_{seed}}. \tag{3}$$

The testcase-specific weights α, β and γ, given as program inputs, combine them into the PPA term

$$P = \alpha \cdot TNS_{norm} + \beta \cdot Power_{norm} + \gamma \cdot WL_{norm}. \tag{4}$$

D is the average Manhattan displacement per cell in site units, and R is runtime normalized against a per-testcase reference. Both are normalized against the seed in the same way. The final score per design is

$$S = 1000 \cdot P - 50 \cdot D - 30 \cdot R. \tag{5}$$

Submissions run in Docker with 8 vCPUs, 32 GB of RAM and one NVIDIA A100 GPU (CUDA 11.8).

3 Proposed Method

3.1 Overall flow

The flow applies the transformations in a fixed order inside a loop and keeps iterating while the contest score S improves (Fig. 1). Each pass ends with legalization and wirelength optimization, so every intermediate result is a legal, scorable solution. When S stops improving, the flow outputs the best DEF and changelist found so far.

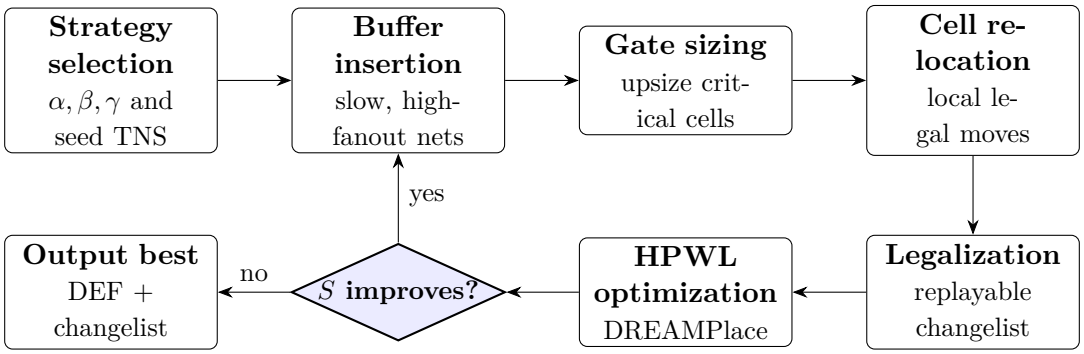


Figure 1: Overall optimization flow.

3.2 Strategy selection

The weights α, β and γ change per testcase, so a single fixed recipe cannot suit every design. Before optimizing, the flow computes the seed's setup TNS over timing endpoints and the ratio $r = N_{cells}/|TNS|$. It then picks one of four strategies (Table 1).

3.3 Buffer insertion

Candidate selection. Every driver-to-sink net arc reported by the timing engine is a candidate, ranked by

$$score = \max(delay_{rise}, delay_{fall}) \cdot w_{timing} + 0.1 \cdot fanout, \tag{6}$$

Table 1: Strategy selection.

Condition	Timing optimization	Wirelength optimization
$r > 10$ and $\alpha > 10(\beta + \gamma)$: timing dominates	Balanced timing and power weights	Skipped, to protect timing
$r > 10$ and $3\alpha < \beta + \gamma$: power and wirelength dominate	Skipped	Full DREAMPlace pass
$r > 10$, other weights	Balanced timing and power weights	Gentle pass: 1 iteration, learning rate 5×10^{-4}, target density 0.70
$r \leq 10$ (severe violations), or TNS unavailable	Aggressive: timing weight 5, power weight 1	Full DREAMPlace pass

where w_{timing} is the timing weight from Section 3.2 and $fanout$ is the driver pin's fanout. The small 0.1 coefficient lets fanout break ties without outweighing delay.

How many nets to buffer. The share of candidates kept depends on how badly the design fails timing. A base share of 45%–95% is set by the fraction of pins with negative slack. It is scaled by 0.95–1.8 according to the average negative slack, and by the timing weight. The result is clamped to 30%–100% of the candidates.

Insertion point. Buffers are inserted on the driver side. This is the simplest choice and immediately relieves the driver's load. It helps less when the loads are clustered far from the driver.

Buffer sizing. Drive strength follows the arc delay: x16 above a delay of 5, x8 above 2, x4 above 0.5 and x1 otherwise. When the power weight exceeds 1, the smallest buffer that is strong enough is preferred.

3.4 Legalization and changelist generation

Every inserted buffer must land on a legal site, or the evaluator cannot replay its command. The legalizer builds a sorted list of occupied intervals for each placement row. It then searches for a free, site-aligned gap near the target: first in the same row, then within the net's bounding box, and finally anywhere in the core.

The NETS section is rewired for each placed buffer and the COMPONENTS count is updated. Commands whose buffer could not be placed are dropped, so the changelist and the DEF always agree. DEF and LEF parsing run in parallel across all CPU cores to keep runtime low.

3.5 Gate sizing

1. **Find critical cells.** Select cells whose worst slack is below a threshold that depends on the timing weight: 0 ps for an aggressive weight, −50 ps for a moderate one and −100 ps for a conservative one.
2. **Rank them.** Score each cell by its worst slack magnitude times the timing weight. The score doubles when the cell degrades slack from input to output, and grows by 1.5× when its fanout exceeds 10. The top $N/10$ cells, scaled by the timing weight, are kept (N = number of cells).
3. **Pick a size.** For each cell, choose the smallest stronger variant with the same function that is expected to fix its slack. Smaller variants cost less power.
4. **Keep or revert.** Keep a resize only if overall timing improves without unacceptable power or hold-time regressions; otherwise revert it or try another size.

3.6 Cell relocation

1. **Pick targets.** Select the cells that contribute most to negative slack.
2. **Generate moves.** Propose a few nearby legal positions, such as a shift toward the driver, a swap with a neighbour or a small offset within the row, all within a few cell widths.
3. **Evaluate.** Estimate each move's effect on critical-path delay and net lengths, including side effects on neighbouring paths.
4. **Check legality.** Reject any move that causes an overlap, leaves the site grid or rows, or touches a fixed macro or blockage.

3.7 Wirelength optimization

Wirelength is reduced with DREAMPlace [6], a GPU-accelerated analytical placer, using HPWL as the usual proxy for Steiner-tree wirelength [2]. We run a short global-placement pass: Nesterov optimizer, weighted-average wirelength model, 200 iterations, learning rate 0.02 and target density 0.99, with the quadratic density penalty off for speed. Section 3.2 decides whether this pass runs and how strongly.

4 Experimental Results

With equal weights ($\alpha = \beta = \gamma = 1$), the flow improved TNS by at least 91.8% and HPWL by at least 7.9% on all six public benchmarks (Table 2, Fig. 2). TNS reached zero

on aes and des. Power fell by 80.0%–91.3% on aes, aes_cipher_top and des, but rose on ac97_top, ariane and pci_bridge32.

All metrics were measured with OpenROAD using the contest evaluation script. Improvements are relative to the original seed placement provided with each testcase.

Table 2: TNS, total power and HPWL before and after optimization.

	TNS (ps)		Power (W)		HPWL (DBU)	
Testcase	Before	After	Before	After	Before	After
ac97_top	−198,213.16	−8,049	1.32×10^{-1}	1.53×10^{-1}	4.08×10^{5}	3.49×10^{5}
aes	−6,833.96	0	2.15×10^{-2}	4.31×10^{-3}	3.78×10^{5}	3.48×10^{5}
aes_cipher_top	−13,971.28	−234.71	7.80×10^{-2}	6.76×10^{-3}	7.68×10^{5}	4.99×10^{5}
ariane	−3,131.37	−257.61	2.86×10^{-2}	4.61×10^{-2}	1.40×10^{7}	3.47×10^{6}
des	−2,049.19	0	8.85×10^{-3}	1.34×10^{-3}	1.17×10^{5}	7.21×10^{4}
pci_bridge32	−389,996.69	−22,978.46	1.04×10^{-1}	1.57×10^{-1}	8.74×10^{5}	7.04×10^{5}

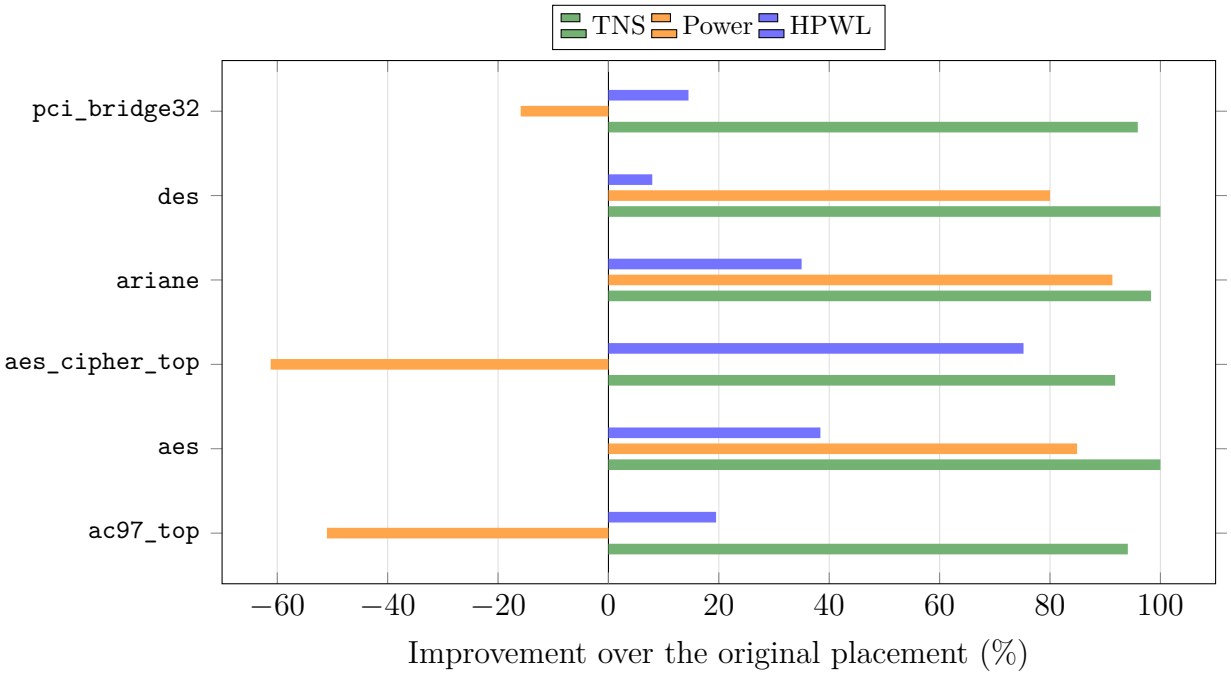


Figure 2: Improvement per metric on the six public benchmarks ($\alpha = \beta = \gamma = 1$). Negative values are regressions.

Discussion. Power rose on ac97_top, ariane and pci_bridge32. ac97_top and pci_bridge32 started with by far the worst TNS of the set (−198 ns and −390 ns), so timing repair inserted many buffers, and each buffer adds leakage and switching power. ariane is the largest design, with 105,730 cells. Under equal weights the flow therefore traded power for a TNS gain of more than 90%. A higher β would make the strategy selector (Section 3.2) buffer less aggressively on these designs. The largest HPWL reduction, 75.2%, was on ariane, the largest design.

5 Conclusion

This project built a closed-loop incremental placement optimizer that combines buffer insertion, gate sizing, cell relocation and wirelength optimization, and iterates while the contest score improves. Under equal weights it cut TNS by 91.8%–100% and HPWL by 7.9%–75.2% on all six public benchmarks while keeping every solution legal and replayable. Power improved on three designs and regressed on three, which reflects the inherent trade-off between timing repair and energy. The method received an Honorable Mention in the domestic round of the 2025 CAD Contest.

Three directions would improve it further:

- **Power-aware buffering.** Include the power cost of each buffer in the candidate score, so that designs with large TNS do not pay for timing with large power increases.
- **Better insertion points.** Place buffers near clusters of loads rather than only at the driver.
- **Incremental timing.** Re-run STA inside the loop, for example with a GPU timer such as INSTA, so each iteration ranks candidates on up-to-date slack.

6 Plan Management and Teamwork

6.1 Plan management

The contest problem was released around 10 March 2025, and the final solution was submitted on 12 September 2025. Most of the work was coding. Each week we ran the public testcases and reviewed the results with Prof. Mak, who suggested improvements and explained the relevant EDA background.

We first built a machine-learning approach: graph neural network (GNN) models for gate sizing, relocation and buffer insertion, combined into an ensemble. It did not give good results by the beta submission, so in August we switched to the conventional, rule-based method in this report.

6.2 Teamwork

The coding was split equally between the two team members. We discussed problems over messages as they came up, shared relevant papers and ran experiments together.

Table 3: Project timeline (2025).

Dates	Work	Milestone
Mar 10 – Apr 11	Literature survey and background study	
Apr 12 – Apr 19	OpenROAD environment setup and testing	
Apr 20 – Apr 26	Testing 2024 Problem C scripts; survey of placement legalization	
Apr 27 – May 10	GNN model testing on 2024 testcases	May 10: registration
May 11 – May 17	Gate-sizing dataset construction	
May 18 – May 31	GNN gate-sizing model training, plus legalization	
Jun 1 – Jun 7	Final exams	
Jun 8 – Jun 14	Model weight tuning; alpha submission (sizing only)	Jun 13: alpha test
Jun 15 – Jun 28	Relocation model training	
Jun 29 – Jul 12	Buffer-insertion model training	
Jul 13 – Jul 26	Model ensembling	
Jul 27 – Aug 2	Beta submission	Aug 1: beta test
Aug 3 – Aug 9	Rule-based gate sizing	
Aug 10 – Aug 16	Rule-based buffer insertion	
Aug 17 – Aug 23	Rule-based cell relocation	
Aug 24 – Aug 30	DREAMPlace wirelength optimization	
Aug 31 – Sep 6	Full-flow integration and testing	
Sep 7 – Sep 12	Final submission	Sep 12: final

References

[1] S. Sahni and T. Gonzalez, "P-complete approximation problems," *Journal of the ACM*, vol. 23, no. 3, pp. 555–565, Jul. 1976.

[2] M. R. Garey and D. S. Johnson, "The rectilinear Steiner tree problem is NP-complete," *SIAM Journal on Applied Mathematics*, vol. 32, no. 4, pp. 826–834, Jun. 1977.

[3] W.-N. Li, "Strongly NP-hard discrete gate-sizing problems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 8, pp. 1045–1051, Aug. 1994.

[4] W. Shi, Z. Li and C. J. Alpert, "Complexity analysis and speedup techniques for optimal buffer insertion with minimum cost," in *Proc. Asia and South Pacific Design Automation Conference (ASP-DAC)*, Jan. 2004, pp. 609–614.

[5] T. Ajayi and D. Blaauw, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Proc. Government Microcircuit Applications and Critical Technology Conference (GOMACTech)*, 2019.

[6] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in *Proc. 56th ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.

[7] Y.-C. Lu, R. Liang, W.-H. Liu and H. Ren, "Incremental placement optimization beyond detailed placement: Simultaneous gate sizing, buffering, and cell relocation," 2025 CAD Contest at ICCAD, Problem C, rev. 2025-09-03.